



Show Us How You Build

Savomart Full Stack Engineer · 48-hour hackathon

Overview

Every new Savomart store starts with a location decision. Today, our **Business Development (BD)** executives drive around the city, spot vacant properties using their own judgement, and only then do we check the rent, the site and the catchment. It works, but it is slow, hard to scale, and depends on who happens to drive down which street.

We want to flip that around: **let the data point BD teams to the right areas before anyone leaves the office**, then carry every property from first sighting to final decision in one place, with the field survey teams working in the same loop.

Your task is to build the **first version of this expansion intelligence platform** (working title **Savo SiteScout**; feel free to name it) for our **Chennai** region.

This is a real product with real users, not a toy. It is **deliberately open-ended**. We describe *what* each user needs; the *how* (scoring, workflows, data model, UX) is yours to design. Aim for **a working end-to-end product that does the core loop well**, and add extras only once that loop works.

Time + AI Policy

- **Time: 48 hours, Monday, 28 September 2026, 9:00 AM IST → Wednesday, 30 September 2026, 9:00 AM IST.** Your Git commit history is the source of truth for timing.
- **AI tools (Claude, ChatGPT, Cursor, Copilot, Claude Code, etc.):** fully allowed and encouraged. We'd rather see you use AI well than pretend you didn't. **Disclose in your README which AI tools you used and how, and share your AI chat sessions / history** (see Deliverables).
- **Open-source / libraries:** use them freely. Cite the substantial ones in your README. For data, see *Data Sources* below.

Tech Stack

- **Backend:** Python, using **FastAPI** or **Django**.
- **Frontend:** **React**.
- **Database:** your choice. This problem is heavily location-based, so choose with that in mind.
- **Map library:** your choice (Leaflet / MapLibre + OpenStreetMap are fine free options; Mapbox / Google Maps also fine if you bring your own key).
- **LLM:** bring your own. Any provider works, including free tiers or a local model. Keep the provider swappable via configuration, and **never commit API keys**.
- **UI library:** your choice.

We're not grading on which library you picked. We're grading on how you used it.

The Problem

Build a web application that takes Savomart's expansion process from **“which area?”** through **“which property?”** to **“is the catchment right?”**, shared by four kinds of users:

Persona	What they need from the product
BD Manager	Explore areas of Chennai without visiting them, see which are worth scouting, direct executives, review properties and decide what moves forward, and request catchment studies.
BD Executive	Know where to scout, then onboard properties they find in the field quickly from a phone.
Survey Manager	Receive catchment study requests, plan and split the work, assign it to the team and track progress.
Survey Executive	See their assigned work and capture lane-level ground data, mostly on a phone, on the street.

Color palette (mandatory): your app must use Savomart's brand colors, **#782B90** (purple) and **#FFF200** (yellow), as its primary visual identity. How you apply them is up to you.

Focus region (mandatory): Chennai (Greater Chennai / Chennai Metropolitan Area). Everything you build and demo should work on real Chennai geography.

Must-Have: Milestones

Build in this order. **Each milestone should be demoable on its own**, and a complete M1-M3 loop beats a polished M1 with nothing after it. The prompts under each milestone are questions for you to answer in your design. They are **not** hidden requirements, and there is no single right answer.

Users and roles: keep this simple. Seeded demo users for the four personas, or a simple role switcher, is enough (document how to use it). Spend your time on M1-M3, not on building an auth system. Each persona should still see a view shaped around *their* job.

M1 Area Intelligence: virtual analysis

- A BD Manager selects any area of Chennai on an **interactive map (mandatory)** by **pincode**, by **locality name**, or by **picking grid cells** on the map.
- They trigger a **virtual analysis** that produces an **Area Fitness Report**: what the area is like (its people, homes, businesses, amenities, competition, and how close existing Savomart stores are).
- The report **rates how well the area fits Savomart**, explains *why*, and suggests **where within the area** executives should scout first.
- Reports are saved, can be revisited and compared, and are clearly timestamped with the data they used.

Things to think about

- What makes an area a good fit for a neighbourhood grocery store? Which signals can you actually get from public data?
- Why should a BD Manager trust your rating? Can they see how it was reached?
- Where does AI add value here, and how do you stop it from inventing numbers?
- Analysis may take a while. What does the user see while it runs, or if it fails halfway?

M2 Property Scouting and Evaluation

- A BD Manager can direct executives to scout the areas or hotspots from M1.
- A BD Executive **onboards a property from the field**: its location, the commercial and physical details you think matter, and photos. It must work well on a phone.
- Each new property is **automatically evaluated** by combining public data around it with what the executive captured, and the evaluation returns insights, risks and a recommendation.
- The BD Manager moves properties through a **pipeline of stages** that you design (for example, asking an executive to proceed to the next stage, or rejecting a property), with a clear history of who did what and why.

Things to think about

- What does the BD Manager need to see to decide in 30 seconds?
- How do you handle bad field input, like a wrong pin, missing rent or duplicate properties?
- Should a property's evaluation change when new data arrives later (for example, a catchment study)?

M3 Catchment Study: survey operations

- A BD Manager can request a catchment study against **a property** or against **an area** that has been virtually analysed.
- The Survey Manager sees incoming requests, **breaks each one into smaller units of work**, and assigns them to executives.
- A Survey Executive sees their assignments and **captures data lane by lane**. You decide what is worth capturing.
- Managers can see progress, and completed data **rolls up into catchment insights** linked back to the property or area.
- **Reuse**: if a property is scouted inside an area that already has a catchment study, that data should be reused rather than surveyed again.

Things to think about

- How do you split a catchment into fair, non-overlapping chunks of work?
- When is an existing study "close enough" and "fresh enough" to reuse? Where do you draw the line?
- Lanes often have weak mobile network. What happens to a half-filled survey?

Stores API (live)

Fetch Savomart's operational stores (useful for proximity, coverage and cannibalisation) from our internal service:

```
curl --location \
  'https://internal-service.savomart.in/bridge/api/store/list?is_operational=True' \
  --header 'X-cron-token: savo-bridge-cron-secret' \
  --data ''
```

Data Sources: OpenStreetMap + Indian Government Open Data

Build your analysis on **OpenStreetMap (and its ecosystem of tools)** and **Indian government open data**. The links below are starting points, not a checklist; which datasets you pick and how you combine, clean and interpret them is part of the challenge. Cite everything you use in your README. Where no public data exists (e.g. rents), sensible mock data is fine. **Label it clearly** in the UI and README.

Source	What it can give you	Link
OpenStreetMap (Geofabrik extract)	Roads/lanes, shops, supermarkets, schools, hospitals, offices, transit. Southern Zone covers Tamil Nadu.	download.geofabrik.de
Overpass API / Overpass Turbo	Query OSM features for a specific area on demand.	overpass-turbo.eu
Nominatim	Geocoding and reverse geocoding on OSM data (respect the usage policy).	nominatim.org
Pincode boundaries (OGD India)	All-India pincode boundary GeoJSON.	data.gov.in
Pincode directory (OGD India)	Post offices by pincode, with latitude / longitude.	data.gov.in
OSRM (OSM routing)	Road-network routing and travel times on OSM data.	project-osrm.org
Census of India	Ward / town-level population, households and household amenities.	censusindia.gov.in
Tamil Nadu OGD portal	State-level open datasets published by Tamil Nadu government departments.	tn.data.gov.in
Bhuvan (ISRO / NRSC)	Indian geoportal with thematic layers such as land use / land cover.	bhuvan.nrsc.gov.in

Bonus: Nice-to-Have

Only once M1-M3 work end to end. Pick what excites you. **One well-shipped bonus beats three half-built ones.**

- **Conversational analyst.** An AI assistant that a BD Manager can ask questions across areas, properties and surveys (e.g. *"compare Velachery and Tambaram for our next store"*), grounded in your data.
- **City-wide opportunity map.** Proactively surface high-potential, un-scouted pockets across Chennai without the manager having to pick an area first.
- **Offline-first field capture.** Survey and scouting flows that keep working without network and sync later.
- **Shareable decision pack.** A one-click export (PDF or similar) of a property's full case for leadership sign-off.
- **Notifications / activity feed** so each persona knows what needs their attention.
- **Deploy the app** somewhere accessible (Vercel, Render, Railway, Fly.io, AWS, etc.) with seeded demo data, and share the URL.
- **Meaningful automated tests / CI** around the parts that matter most.

How We'll Evaluate

We'll look at the whole product, across:

- **Product thinking:** did you understand the users and design flows around their real jobs?
- **Frontend and UX:** map interactions, role-specific experiences, mobile field flows, loading and error states.
- **Backend and API design:** structure, workflows and state, background processing, access control, error handling.
- **Data and database:** data model, spatial querying, how you ingested and cleaned public data.

- **AI integration:** useful, grounded, explainable, and not a wrapper that makes up numbers.
- **Engineering hygiene:** code quality, README, commit history, and honest trade-offs.

Deliverables

A **single private GitHub repository**, named `savomart-fullstack-hackathon-2026`, with **mohammed.hafiz@ebono.com added as a collaborator**, containing:

1. **All source code:** backend, frontend, data-ingestion scripts and configuration. Everything needed to run your project should live in the repo (large raw datasets can be fetched by a script instead).
2. **A well-documented README.md** covering at minimum: setup instructions to run the project locally end to end; demo credentials for each role; an architecture overview; your data model and schema with a brief explanation; data sources used and how you processed them; how your scoring and AI work (and how you keep them grounded); tech and library choices with short justifications; design decisions and trade-offs; known issues and what you'd improve with more time; how and where you used AI tools; and the link to your video demo.
3. **AI chat sessions / history:** export or share the AI conversations you used while building (e.g. shared chat links, exported transcripts, or Cursor / Claude Code session logs), either in an `/ai-sessions` folder or linked from the README. We read these to understand how you think and work with AI, not to penalise heavy use.
4. **A short video demo (3 to 5 minutes):** walk through the core loop across the personas and briefly explain your architecture and key decisions. **Upload to Google Drive** with sharing set to *"Anyone with the link can view"*, and include the link in your README.
5. **Proper commit history** that reflects work spread across the 48 hours. A single end-of-window commit is a red flag.

Submission

- **Email** the GitHub repo link and the Google Drive video link to **mohammed.hafiz@ebono.com**.
- **Subject:** Savomart Full Stack Hackathon – <Your Full Name>.
- **Deadline: Wednesday, 30 September 2026, 9:00 AM IST.** Late submissions will be reviewed at lower priority.
- **Questions?** Reach out anytime: email **mohammed.hafiz@ebono.com** or ping/message on WhatsApp **+91 8220268265**.

A Few Final Notes

- We're evaluating **how you think, design, and ship**, not whether you ticked every box. A working M1–M3 loop with a thoughtful README will beat a half-broken feature dump every time.
- **We've intentionally left the big decisions open:** how to score an area, what the pipeline stages are, how to split a survey, what to capture in a lane. We want *your* solutions and perspective, backed by reasoning. Tell us why you chose what you chose.
- **Bring your own creativity.** Wherever you can make the experience more delightful, solve a problem more elegantly, or surprise us with a thoughtful detail, please do.
- **Asking clarifying questions is encouraged, not penalised.** If any requirement is ambiguous, reach out. That's a positive signal about how you'd work with the team.

Good luck. We're looking forward to seeing what you build.